

MULTI-PURPOSE IoT BASED SURVEILLANCE ROBOT

A

Major Stage-2 Project Report

Submitted in the Partial fulfilment of the
Academic Requirements for the Award of the Degree of

Bachelor of Technology

In

Electronics and Communication Engineering

Submitted By

P. VENKATESH SAGAR

22AG1A0446

Under the esteemed guidance of

Mr. B. GIRIRAJU

Professor



**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**

ACE Engineering College

(NBA ACCREDITED B. TECH COURSES ECE, EEE, CSE, MECH, CIVIL)

(NAAC “A” GRADE)

An Autonomous Institution

Ankushapur (V), Ghatkesar (M), Medchal (Dist)-501 301

2025-2026



ACE ENGINEERING COLLEGE

An Autonomous Institution

(Approved by AICTE, New Delhi & Affiliated to JNTUH) Accredited with NBA & NAAC 'A' Grade



Department of Electronics and Communication Engineering

CERTIFICATE

This is to certify that the major project entitled **“Multi-Purpose IoT Based Surveillance Robot”** done by

P. VENKATESH SAGAR

22AG1A0446

Department of Electronics and Communication Engineering is a record of Bonafide work carried out by them. This major project is done as partial fulfilment of obtaining Bachelor of Technology degree to be awarded by **JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY, HYDERABAD** during the academic year 2025-2026.

Mr. B. GIRIRAJU

Professor and Guide

Dr. Y. Chakrapani

Professor and DEAN

ECE Department

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

I would like to express our deepest appreciation and gratitude to **Prof .B.GIRIRAJU** for his invaluable guidance, constructive criticism, and encouragement during the course of this project.

Thanks to our Head of the Department **DR.Y.CHAKRAPANI** for being uniformly excellent advisors. They were always open, helpful, and provided strong broad ideas.

I deeply indebted to **Dr.N.SUDHAKAR REDDY** Principal of the college , for providing the necessary opportunities for the completion of the project.

I am grateful to **Prof. Y. V. GOPALA KRISHNA MURTHY**, for his moral support and for providing facilities.

I am thankful to the teaching and non-teaching staff members of the Electronics and Communication Engineering Department who helped directly or indirectly for successful completion of the project.

I am very grateful to my family and friends for their constant support and encouragement during the project period.

With Regards,

P. VENKATESH SAGAR

22AG1A0446

ABSTRACT

In this project, a multi-purpose IoT-based surveillance robot has been designed and developed to perform monitoring operations in domestic and sensitive areas. The system aims to reduce human effort and minimize human error by utilizing robotic automation for surveillance tasks. The robot is capable of navigating the environment and capturing real-time audio and video data, which is transmitted to the user for remote monitoring. Control of the robot is achieved through Internet of Things (IoT) technology using a mobile phone or laptop, enabling both manual and automated operation modes. An ESP32 microcontroller is used to process sensor data and control the robot's movement and overall behavior.

Wireless cameras provide live video streaming during both daytime and nighttime conditions. A smoke sensor with MQTT protocol enables real-time fire hazard alerts. A flame sensor detects fire in the surrounding environment. A GPS module provides live location tracking of the robot. A metal detection sensor identifies the presence of metallic objects such as weapons or explosives. The experimental results demonstrate reliable multi-purpose surveillance performance, validating the system's suitability for security, disaster response, and defense applications. Further enhancement of the project can extend its usage to large-scale and defense surveillance systems.

Keywords: ESP32, IoT, Surveillance Robot, Smoke Sensor, Flame Sensor, GPS, Metal Detector, MQTT

CONTENTS

Certificate	ii
Acknowledgement	iii
Abstract	iv
Contents	v
List of Figures	vi
CHAPTERS	Page no
1. INTRODUCTION	1
1.1 Project Overview	1
1.2 Block Diagram	1
1.2.1 Block Diagram Description	2
1.3 Components	2
1.3.1 ESP32 Microcontroller	2
1.3.2 ESP32-CAM Module	3
1.3.3 L293D Motor Driver	4
1.3.4 Ultrasonic Sensor	5
1.3.5 Smoke Sensor (MQ-2 / MQTT)	5
1.3.6 Flame Sensor	6
1.3.7 GPS Module (NEO-6M)	7
1.3.8 DC Motors and Wheels	8
2. LITERATURE SURVEY	9-10
3. DESIGN AND IMPLEMENTATION	11
3.1 Technical Description	12
3.2 Working	13
3.3 Procedure	14
4. SYSTEM ANALYSIS	15
4.1 Existing System	15
4.2 Proposed System	16

4.3 Working Principle	17
4.4 Advantages and Disadvantages	18
4.5 Circuit Working	18
5. MODULES	19
5.1 ESP32 Board	19
5.2 Smoke Sensor (MQ-2 / MQTT)	25
5.3 Flame Sensor	27
5.4 GPS Module (NEO-6M)	29
5.5 ESP32-CAM Module	33
5.6 Robot Chassis	35
5.7 Buck Converter (DC-DC Step-Down)	37
5.8 Battery	39
6. SOFTWARE TOOLS	42
6.1 Arduino IDE Overview	43
6.2 MQTT Protocol / Cayenne	44
6.3 Installation and Upload Steps	45
SOURCE CODE	46-56
7. RESULTS	57
7.1 Model Diagram of the Project	57
7.2 ON STATE and OFF STATE	58
7.3 Advantages	58
7.4 Limitations	59
7.5 Applications	59
8. CONCLUSION AND FUTURE SCOPE	60
8.1 Conclusion	60
8.2 Future Scope	61
BIBLIOGRAPHY	62

LIST OF FIGURES

Page no

1.2	Block Diagram of Surveillance Robot	1
1.3.1	ESP32 Development Board	3
1.3.2	ESP32-CAM Module	3
1.3.3	L293D Motor Driver IC	4
1.3.4	Ultrasonic Sensor HC-SR04	5
1.3.5	Metal Detector Sensor	5
1.3.6	MQ-2 Smoke Sensor	6
1.3.7	Flame Sensor Module	7
1.3.8	GPS Module NEO-6M	7
1.3.9	DC Motors	8
4.1	Existing System Block Diagram	15
4.2	Schematic Circuit Diagram	16
5.1	Esp32 board	19
5.2	Esp32 Specifications	20
5.3	Esp32 Board Guide	21
5.4	MQT Smoke sensor	27
5.5	Flame Sensor Module	29
5.6	GPS Neo 6m Module	30
5.7	GPS module	31
5.8	Esp32 Cam module	35
5.9	Robot Chassis	37
5.10	Buck Converter	39
5.11	Battery	41
6.1	Arduino IDE Logo	42
6.1.1	Arduino IDE Interface	43
7.1	Working Model of the Project	56
7.2	ON STATE	57

CHAPTER 1

INTRODUCTION

1.1 PROJECT OVERVIEW:

The Multi-Purpose IoT Based Surveillance Robot is designed to improve security monitoring in domestic, commercial, and sensitive environments by automating surveillance tasks using embedded systems and Internet of Things (IoT) technology. The robot integrates multiple sensors and communication modules to deliver comprehensive, remotely controllable surveillance. An ESP32 microcontroller forms the central processing unit of the entire system.

The system allows users to control the robot remotely via a smartphone or laptop through IoT connectivity. An ESP32-CAM module provides live video streaming for real-time visual monitoring. An ultrasonic sensor enables autonomous obstacle detection. A metal detector sensor identifies hidden metallic objects. A smoke sensor connected via MQTT protocol detects hazardous smoke or gas. A flame sensor detects fire. A GPS module provides real-time location tracking of the robot, making it suitable for large-area patrol and defense applications.

1.2 BLOCK DIAGRAM:

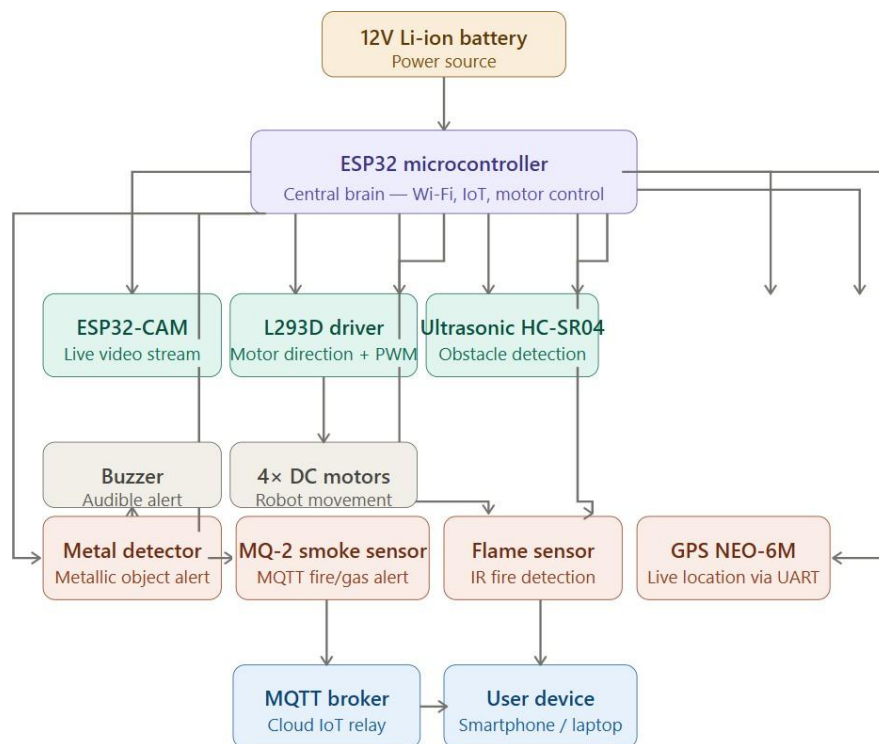


Figure 1.2 Block Diagram of Surveillance Robot

1.2.1 BLOCK DIAGRAM DESCRIPTION:

The block diagram of the multi-purpose IoT surveillance robot begins with the ESP32 microcontroller, which forms the central brain of the system. The ESP32-CAM module is interfaced with the ESP32 and independently hosts a web server to deliver live video to the user's smartphone through a browser. The L293D motor driver is connected between the ESP32 and the four DC motors, enabling bidirectional control for navigation. The ultrasonic sensor interfaces with the ESP32 digital pins and continuously measures distance to obstacles, halting the motors automatically when an obstacle is detected within a threshold distance.

The metal detector sensor outputs a digital HIGH signal when metal is detected, triggering a buzzer alert. The MQ-2 smoke sensor connects to the MQTT broker to publish real-time smoke and gas concentration data to the user's device. The flame sensor directly signals the ESP32 upon detecting infrared radiation from fire. The GPS module (NEO-6M) communicates with the ESP32 via UART and sends latitude/longitude coordinates for live tracking. All modules are powered from a 12V Li-ion battery, which also supplies regulated voltage to the ESP32 and sensor circuits.

1.3 COMPONENTS:

1.3.1 ESP32 Microcontroller

The ESP32 is a powerful, low-cost, low-power system-on-chip (SoC) developed by Espressif Systems. It integrates a dual-core Xtensa LX6 processor running at up to 240 MHz, along with built-in Wi-Fi (802.11 b/g/n) and Bluetooth (Classic and BLE). The ESP32 features 34 programmable GPIO pins, multiple communication interfaces including SPI, I2C, I2S, and UART, 12-bit ADC channels, and DAC outputs. It operates at 3.3V and can be powered via USB or external supply. The ESP32 replaces the Arduino in this project and serves as the sole central controller, offering significantly higher processing power and native wireless connectivity.

Function: In this project, the ESP32 is responsible for: receiving IoT control commands via Wi-Fi, driving the L293D motor driver for robot movement, reading ultrasonic sensor data for obstacle avoidance, receiving metal detector and flame sensor signals, publishing smoke sensor data over MQTT, and receiving GPS NMEA data for location tracking. Its dual-core architecture allows sensor processing and communication tasks to run concurrently without performance loss.



Fig 1.3.1: ESP32 Development Board

1.3.2 ESP32-CAM Module

The ESP32-CAM is a small-sized camera module based on the ESP32 chipset. It features an OV2640 camera capable of capturing images and video at resolutions up to 1600x1200 (UXGA). The module includes a microSD card slot for local storage and an onboard LED flash for low-light conditions. It can host its own HTTP web server to stream MJPEG video directly to a browser without requiring any additional server infrastructure. The ESP32-CAM communicates with the main ESP32 controller via UART for coordinated system operation.

Function: The ESP32-CAM captures live video of the robot's surroundings and streams it to the user's smartphone or browser via a web server. This enables continuous real-time monitoring during all surveillance operations. The onboard LED flash can be toggled remotely for night-vision capability in low-light environments.



Figure 1.3.2 ESP32-CAM Module

1.3.3 L293D Motor Driver

The L293D is a popular dual H-bridge motor driver IC that can independently control two DC motors. It supports motor supply voltages between 4.5V and 36V and delivers up to 600mA per channel. Built-in protection diodes prevent back-EMF damage. The IC accepts TTL-compatible logic signals

from the ESP32 and amplifies them to drive the motors at full power. Direction is controlled by two digital pins per motor and speed is controlled via PWM.

Function: In this robot, the L293D interfaces between the ESP32 and four DC motors. It receives PWM and direction signals from the ESP32's GPIO pins and drives the motors for forward, backward, left, and right movements using differential steering. The driver isolates the high-current motor circuit from the low-current ESP32 logic, protecting the microcontroller from damage.

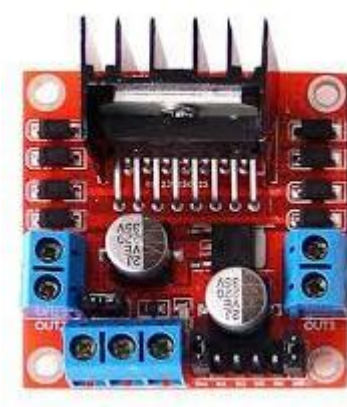


Figure 1.3.3 L293D Motor Driver IC

1.3.4 Ultrasonic Sensor (HC-SR04)

The HC-SR04 ultrasonic sensor measures distances by emitting a 40kHz ultrasonic burst and measuring the time taken for the echo to return. It can detect objects from 2 cm to 400 cm with an accuracy of 3 mm. The sensor has four pins: VCC (5V), Trig (trigger input), Echo (echo output), and GND. A 10-microsecond pulse on the Trig pin initiates a measurement. The Echo pin remains HIGH for a duration proportional to the round-trip travel time of the ultrasonic pulse.

Function: The ultrasonic sensor is mounted at the front of the robot chassis and continuously monitors the distance to obstacles. When the measured distance falls below a programmable threshold (e.g., 20 cm), the ESP32 halts motor operation automatically, preventing collisions. This enables semi-autonomous navigation in environments where the user cannot see all obstacles in the video feed.



Figure 1.3.4 Ultrasonic Sensor HC-SR04

1.3.6 Smoke Sensor (MQ-2 / MQTT)

The MQ-2 is a gas and smoke sensor that can detect LPG, smoke, alcohol, propane, hydrogen, methane, and carbon monoxide. It uses a tin dioxide (SnO_2) sensing element whose electrical resistance decreases when combustible gas is present. The sensor outputs both a digital threshold signal and an analog concentration value. In this project, the smoke sensor data is published over the MQTT (Message Queuing Telemetry Transport) protocol, allowing the ESP32 to act as an MQTT client that sends real-time sensor readings to a cloud broker. The user's phone subscribes to this MQTT topic and receives instant alerts whenever smoke or hazardous gas is detected.

Function: The smoke sensor is mounted on the robot body. As the robot patrols an area, it continuously monitors the air for smoke or combustible gases. Upon detection, the ESP32 publishes an alert message to the MQTT broker, which is immediately delivered to the user's subscribed mobile application. This enables remote fire and gas hazard detection without requiring the user to actively monitor the video feed.



Figure 1.3.6 MQ-2 Smoke Sensor

1.3.7 Flame Sensor

The flame sensor detects the presence of fire or any intense light source by sensing the infrared (IR) radiation emitted by flames. It uses a photodiode that is specifically sensitive to IR wavelengths in the 760 nm to 1100 nm range, which corresponds to the spectrum of fire. The sensor outputs a digital LOW signal when a flame is detected within its detection angle (approximately 60 degrees) and detection range (up to 1 meter). It is compact, fast-responding, and operates on 3.3V to 5V.

Function: The flame sensor is positioned on the front of the robot chassis. As the robot navigates through an environment, it continuously monitors for any open flames or fire sources. Upon detection, the flame sensor sends a LOW digital signal to the ESP32, which triggers both a local buzzer alarm and an MQTT notification to the user's device. This makes the robot valuable for fire detection in warehouses, industrial sites, and disaster response scenarios.



Figure 1.3.7 Flame Sensor Module

1.3.8 GPS Module (NEO-6M)

The NEO-6M is a high-performance GPS module from u-blox that can track up to 22 satellites and lock onto a GPS fix in as little as 1 second under good sky conditions (hot start). It outputs standard NMEA 0183 protocol data including GPGLL and GPRMC sentences, which contain latitude, longitude, altitude, speed, and time information. The module communicates via UART at 9600 baud by default and operates at 3.3V to 5V. It includes an onboard ceramic patch antenna and can be used with an external active antenna for improved reception indoors. In addition to its hardware capabilities, the NEO-6M module includes internal memory and configuration options, allowing it to store certain parameters and maintain performance efficiency. Its compact size, low power consumption, and reliable positioning performance make it a popular choice for projects such as

vehicle tracking systems, smart navigation devices, drone navigation, asset tracking, geographic data logging, and autonomous robotics applications.

Function: The GPS module is mounted on top of the robot chassis for clear sky visibility. The ESP32 reads NMEA data from the GPS module via UART and parses the latitude and longitude coordinates. These coordinates are periodically published to the MQTT broker or displayed on the user's web control interface, enabling live location tracking of the robot. This is especially useful for large-area outdoor patrol, border surveillance, and disaster search-and-rescue operations.

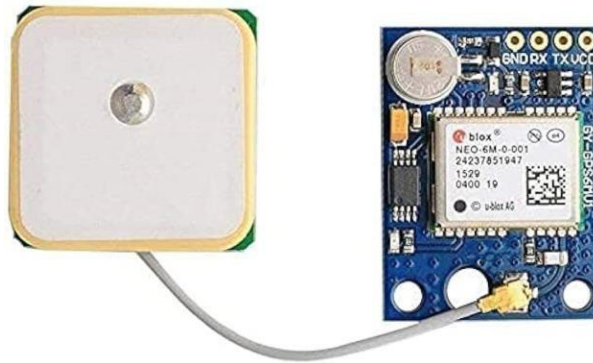


Figure 1.3.8 GPS Module NEO-6M

1.3.9 DC Motors & Wheels

Four dual-shaft DC gear motors are used for robot locomotion. Each motor is a 3V-6V DC motor with a built-in gearbox that reduces speed while increasing torque for reliable movement on flat surfaces. The motors are arranged in pairs on the left and right sides of the chassis. Rubber wheels are attached to each motor shaft to provide traction.

Function: By driving both left and right motor pairs in the same direction, the robot moves forward or backward. By reversing one pair while running the other forward, the robot turns left or right. All motors are driven through the L293D motor driver, which receives PWM speed and direction signals from the ESP32. The motor arrangement enables differential steering for smooth, controllable navigation.

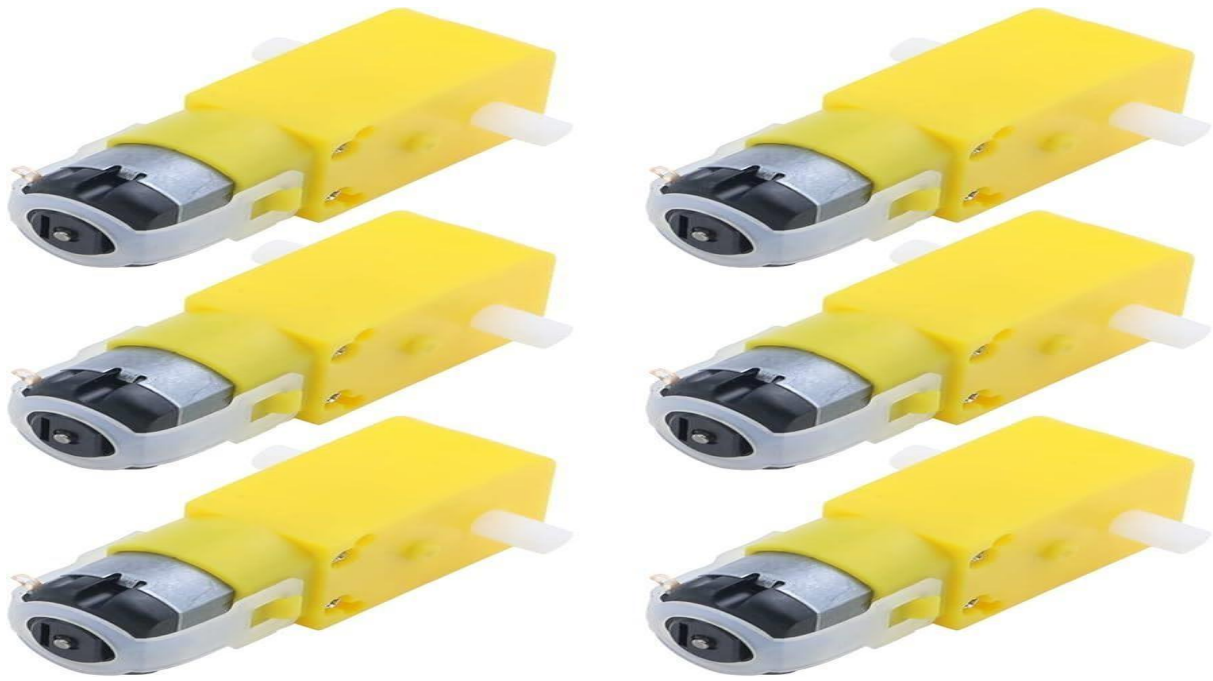


Figure 1.3.9 Dc motors and wheels

CHAPTER 2

LITERATURE SURVEY

[1] Web Controlled Surveillance Robot: An Arduino web-based robotic surveillance device that monitors and records the environment with the help of ultrasonic sensors and a web camera. The system is designed to work in both manual and automated modes. The automated function activates when the user is offline, using the ultrasonic sensor to detect obstacles and initiate avoidance logic. The architecture is closely related to the ESP32-based approach used in this project.

[2] Surveillance Spy Robot with Camera and Metal Detection: This robot monitors surroundings through which the user can control the robot remotely. The live feed from the camera is transmitted to the user's mobile device via a web interface. A metal detector helps detect metal objects and materials. This study demonstrates the value of combining video surveillance with metal detection in a single mobile platform.

[3] Existing surveillance robot systems generally rely on short-range wireless technologies such as RF, Zigbee, and Bluetooth for control and data transmission. Due to limited communication range, operational areas are severely restricted. Most existing designs also lack multi-sensor integration, typically offering either video surveillance or metal detection, but rarely smoke detection, flame sensing, and GPS in a single unified platform.

[4] IoT-based embedded systems for remote monitoring using Wi-Fi (ESP8266/ESP32) achieve extended operational range through internet-based communication, overcoming the limitations of conventional wireless technologies. Real-time data transmission and command reception from any global location is achievable at low cost, making ESP32-based designs the preferred approach for modern surveillance robotics.

[5] MQTT (Message Queuing Telemetry Transport) has been studied extensively as a lightweight publish-subscribe protocol ideal for IoT applications. Research demonstrates that MQTT enables low-latency, reliable sensor data delivery from embedded devices to cloud brokers and end-user applications. Integration of MQTT with environmental sensors such as MQ-2 gas sensors enables effective remote hazard monitoring in industrial and domestic settings.

[6] GPS-enabled mobile robots have been demonstrated for large-area outdoor surveillance and patrol applications. The NEO-6M GPS module combined with MQTT-based location publishing provides a cost-effective solution for real-time tracking of autonomous and semi-autonomous ground vehicles. Studies confirm that GPS accuracy is sufficient for patrol-boundary enforcement and asset tracking in open environments.

[7] Flame and smoke detection robots have been demonstrated in industrial fire safety research. The combination of infrared flame sensors and MQ-type gas sensors provides complementary detection capabilities that reduce false alarms while improving detection reliability. Integrating these sensors into a mobile robot platform significantly extends the detection range compared to fixed sensor installations.

[8] N. Nikhil, P. Shashank Reddy, K. Raju, Associate Prof. R. Srinivasa Rao, "IoT Based Smart Multi-Application Surveillance Robot," International Research Journal of Multidisciplinary Engineering and Technology Sciences (IRJMETS). DOI: <https://www.doi.org/10.56726/IRJMETS42292>. This paper presents a surveillance robot controlled via IoT using an ESP32 camera for live streaming, ultrasonic sensors for obstacle avoidance, and a metal detector for security scanning. The system supports both manual and automatic modes and demonstrates reliable performance in real-world security environments

CHAPTER 3

DESIGN AND IMPLEMENTATION OF SURVEILLANCE ROBOT

3.1 TECHNICAL DESCRIPTION:

The Multi-Purpose IoT Based Surveillance Robot is a cost-effective, integrated system built around the ESP32 microcontroller. The system combines locomotion, real-time video streaming, obstacle detection, hazardous material detection, fire and smoke monitoring, and live GPS tracking into a single mobile platform. All components are interfaced with the ESP32, which serves as the sole controller, processing sensor inputs and executing control commands received via Wi-Fi.

3.1.1 ESP32 Microcontroller

The ESP32 is the core controller of the system. It runs firmware written in embedded C using the Arduino IDE, handling Wi-Fi connectivity, web server hosting, sensor data reading, MQTT publishing, motor control, and GPS data parsing simultaneously using its dual-core architecture.

3.1.2 ESP32-CAM (Camera Module)

The ESP32-CAM is separately programmed to host an MJPEG video stream web server. The main ESP32 receives commands from the user browser alongside the live video feed. The camera's onboard flash LED provides illumination for night surveillance.

3.1.3 Smoke Sensor (MQ-2 / MQTT)

The MQ-2 smoke and gas sensor is connected to an analog input pin of the ESP32. The ESP32 reads the analog concentration value and publishes it to an MQTT broker topic at regular intervals. The user's mobile device subscribes to this topic and receives instant hazard alerts.

3.1.4 Flame Sensor

The flame sensor is connected to a digital input pin of the ESP32. The ESP32 continuously polls this pin and triggers a buzzer and MQTT notification upon flame detection (active LOW output from sensor).

3.1.5 GPS Module (NEO-6M)

The NEO-6M GPS module communicates with the ESP32 via the hardware UART2 interface. The ESP32 parses NMEA GPRMC sentences in real time and publishes latitude/longitude to the MQTT broker, enabling live location tracking on the user's device.

3.1.6 Metal Detector Sensor

The metal detector sensor outputs a digital signal to the ESP32. When metal is detected, the ESP32 activates a buzzer and optionally sends an MQTT notification, alerting the user to the presence of metallic objects at the robot's current GPS coordinates.

3.2 WORKING:

The surveillance robot operates through coordinated interaction between all hardware components, controlled by the ESP32 running embedded C firmware. The system supports two modes: manual IoT-controlled mode and semi-autonomous obstacle-avoidance mode.

3.2.1 User Control via IoT (Manual Mode)

The user accesses a web-based control interface hosted by the ESP32-CAM module through their smartphone browser. Directional commands (forward, backward, left, right, stop) are sent as HTTP GET requests to the ESP32. The ESP32 parses these requests and drives the L293D motor driver accordingly. The live video feed is simultaneously visible on the same web page.

3.2.2 Obstacle Detection and Avoidance

The HC-SR04 ultrasonic sensor continuously measures the distance to any object in the robot's forward path. When the distance falls below 20 cm, the ESP32 automatically overrides user commands and stops the motors, preventing collision. This is handled by the second core of the ESP32 running a dedicated obstacle monitoring task.

3.2.3 Smoke and Flame Detection

The MQ-2 smoke sensor and flame sensor are continuously polled by the ESP32. Upon detecting smoke concentration above a threshold value, or detecting a flame signal, the ESP32 publishes an

alert message to the configured MQTT broker (e.g., broker.hivemq.com). The user's MQTT client application (e.g., MQTT Dash or Cayenne) displays the alert in real time.

3.2.4 GPS Location Tracking

The GPS module provides NMEA sentences over UART. The ESP32 parses the GPRMC sentence to extract current latitude and longitude coordinates. These coordinates are published to a dedicated MQTT topic every 5 seconds, allowing the user to track the robot's location on a map in real time.

3.3 PROCEDURE:

The design and deployment of the Multi-Purpose IoT Surveillance Robot follows a systematic procedure covering hardware assembly, firmware programming, system integration, and testing.

Component Assembly and Wiring

Mount four DC motors at the corners of the robot chassis and attach rubber wheels. Secure the ESP32 development board centrally on the chassis. Connect the L293D motor driver between the ESP32 GPIO pins and the motor terminals. Mount the HC-SR04 ultrasonic sensor at the front of the chassis. Attach the metal detector sensor to the underside of the chassis. Mount the MQ-2 smoke sensor and flame sensor on the top of the chassis for open-air exposure. Mount the GPS module on the top of the chassis for clear sky view. Mount the ESP32-CAM at the front elevated position.

Code Deployment

Open Arduino IDE and install the ESP32 board package via Board Manager. Install required libraries: TinyGPSPlus (GPS parsing), PubSubClient (MQTT), and ESP32Servo. Write the main ESP32 firmware covering: motor control logic, ultrasonic distance measurement, smoke/flame sensor reading, GPS parsing, MQTT client initialization and publishing, and Wi-Fi connection management. Write the ESP32-CAM firmware for video streaming web server. Upload both firmware files to their respective boards.

3.3.1 System Testing and Calibration

Power on the system and connect to the ESP32's Wi-Fi access point. Open the control web page in the browser and verify that live video appears and directional controls respond correctly. Block the ultrasonic sensor manually and verify that motors stop automatically. Expose the smoke sensor to

smoke and verify MQTT alert delivery. Wave the flame sensor near a flame source and verify buzzer and MQTT alert. Verify GPS fix acquisition and coordinate publishing on the MQTT topic.

3.3.2 Optimization and Refinement

Calibrate the ultrasonic obstacle-stop threshold distance. Tune MQ-2 analog threshold for reliable smoke detection without false positives. Adjust GPS NMEA parsing interval to balance location update frequency with processing load. Verify motor PWM duty cycles for smooth speed control. Ensure MQTT broker connection stability with reconnect logic in firmware.

CHAPTER 4

SYSTEM ANALYSIS

4.1 Existing System:

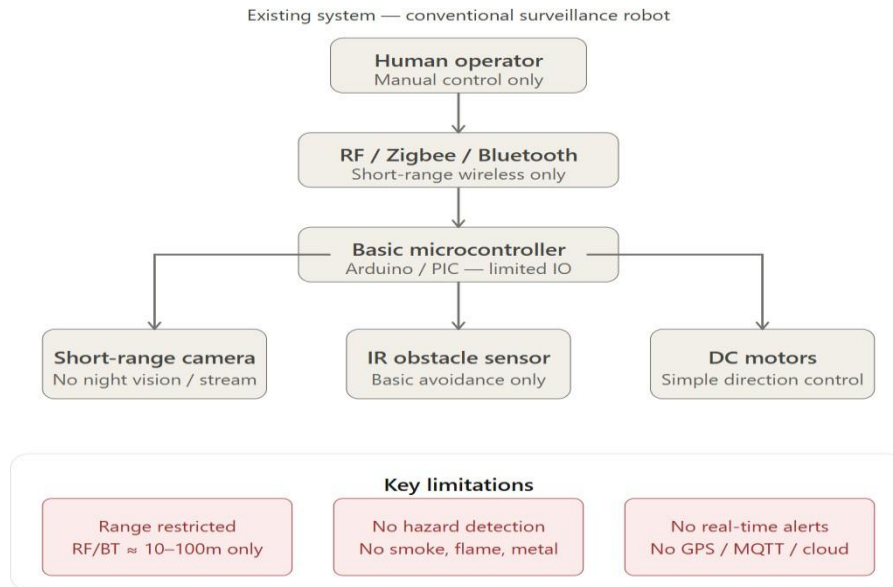


Figure 4.1 Existing System Block Diagram

Existing surveillance robot systems generally rely on short-range wireless communication technologies such as RF, Zigbee, and Bluetooth for control and data transmission. These robots perform basic monitoring tasks and often use short-range wireless cameras to capture visual information from the surrounding environment. Due to limited communication range, the operational area of such systems is severely restricted, making them unsuitable for large premises, open grounds, or remote locations.

Most existing systems offer only one or two sensing capabilities — typically video and basic obstacle avoidance — and lack integrated environmental hazard detection (smoke, flame) and real-time GPS tracking. Manually operated surveillance systems are prone to human error and lapses, leading to security gaps. The absence of cloud-based MQTT communication means that existing systems cannot deliver instant push alerts to remote users.

The circuit diagram illustrates the electronic framework of the proposed surveillance robot. The ESP32 sits at the centre, with the L293D motor driver, ultrasonic sensor, metal detector, smoke sensor, flame sensor, GPS module, and ESP32-CAM all connected to its GPIO and UART pins. The

entire assembly is powered from a 12V Li-ion battery with a buck converter providing 3.3V to 5V regulation for the sensor and ESP32 circuits.

4.2 Schematic Diagram

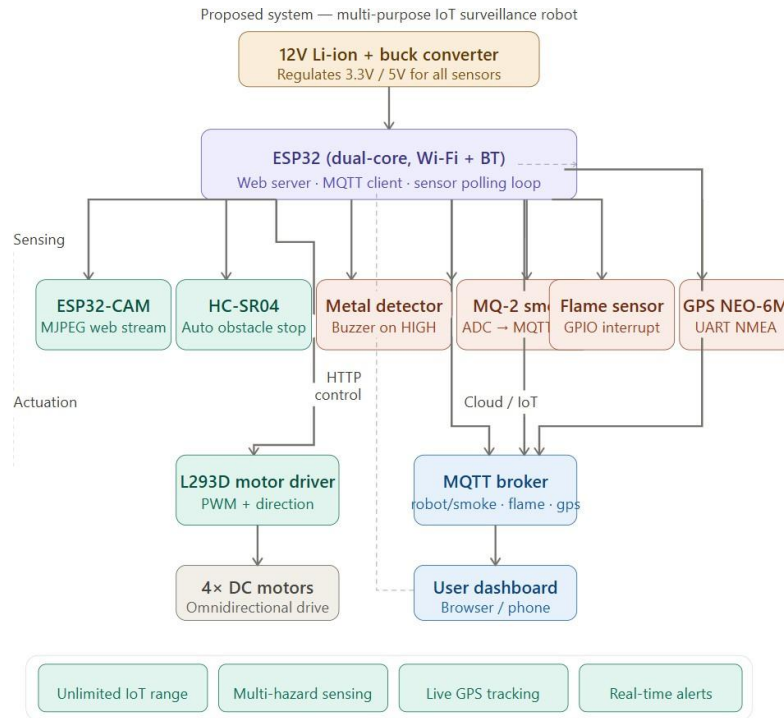


Figure 4.2 Schematic Circuit Diagram of the Proposed System

4.3 Working Principle

4.3.1 System Power Supply The robot is powered by a 12V Li-ion battery. The L293D motor driver draws 12V directly for motor operation. A 5V buck converter regulator supplies power to the ESP32, ESP32-CAM, and most sensors. The GPS module operates at 3.3V, supplied through the ESP32's onboard 3.3V pin.

4.3.2 IoT Control and Motor Operation The ESP32 connects to a Wi-Fi network (or acts as an access point) and hosts a web control page. The user sends directional HTTP requests from their browser. The ESP32 interprets these and drives the L293D with appropriate PWM and direction signals to move the robot as commanded.

4.3.3 Smoke and Flame Monitoring (MQTT) The ESP32 initializes as an MQTT client at startup and connects to a public or private MQTT broker. The MQ-2 smoke sensor analog value is read every second and published to a “robot/smoke” topic. The flame sensor digital pin is monitored

continuously; a flame detection triggers immediate publishing to “robot/flame” topic. The user’s MQTT dashboard subscribes to these topics.

4.3.4 GPS Location Publishing The GPS module sends NMEA data at 1 Hz over UART. The ESP32 parses the GPRMC sentence every 5 seconds and publishes the latitude and longitude to “robot/gps” MQTT topic. The user can view the coordinates on any MQTT-enabled map dashboard for live tracking.

4.3.5 System Reset and Loop The ESP32 main loop runs continuously on Core 1. Core 0 handles Wi-Fi, MQTT, and GPS parsing. The main loop checks: motor commands from HTTP requests, ultrasonic distance measurement (obstacle avoidance override), metal detector digital input (buzzer trigger), and flame sensor digital input. MQTT keep-alive and reconnection are handled automatically by the PubSubClient library.

4.4 Advantages and Disadvantages

4.4.1 Advantages This system provides multi-hazard detection (smoke, flame, metal) in addition to standard surveillance. Unlimited operational range via internet-based IoT and MQTT communication. Real-time GPS location tracking enables large-area monitoring. Live video streaming via ESP32-CAM provides visual situational awareness. Cost-effective — all components are widely available at low cost.

4.4.2 Disadvantages Wi-Fi dependency limits operation in areas with poor network coverage. Battery life is limited under continuous sensor operation and video streaming. GPS fix may take several seconds outdoors and may not work well indoors. MQ-2 smoke sensor may produce false positives from non-hazardous vapors.

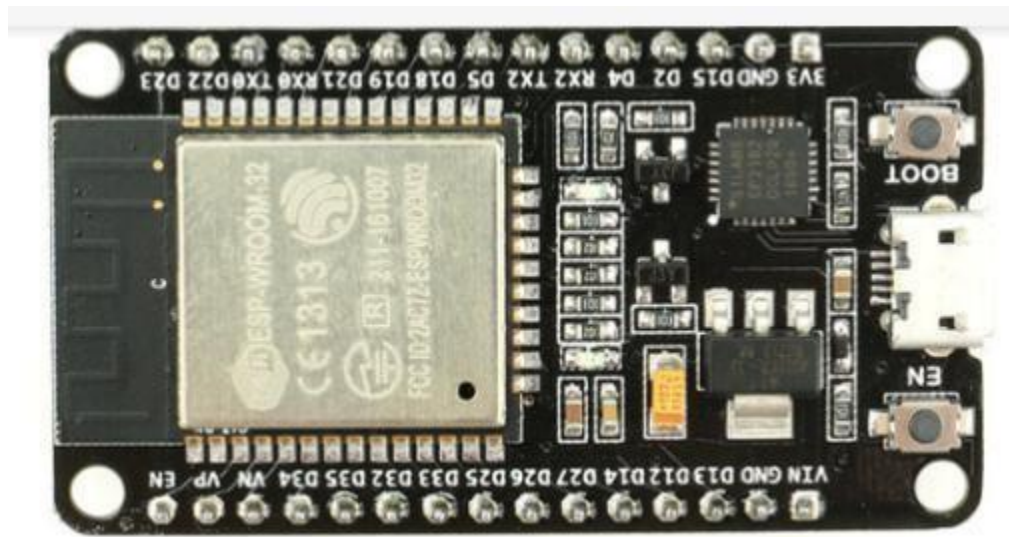
4.5 Circuit Working

1. The ESP32 serves as the central controller. Upon startup, it connects to Wi-Fi, initializes the MQTT client, starts the web server for IoT control, and begins the sensor polling loop.
2. The HC-SR04 ultrasonic sensor continuously measures forward distance. When distance is below threshold, the ESP32 sends a stop command to the L293D, overriding all user inputs.
3. The MQ-2 smoke sensor analog output is read via ADC. When the value exceeds the calibrated threshold, an MQTT alert is published to “robot/smoke” topic and a buzzer is triggered.
4. The flame sensor digital output is monitored on a GPIO interrupt. On detection (LOW output), an MQTT message is published to “robot/flame” topic immediately.
5. The GPS NEO-6M module sends NMEA data via UART2. The TinyGPSPlus library parses GPRMC sentences. Valid coordinate fixes are published to “robot/gps” MQTT topic every 5 seconds.

CHAPTER 5

MODULES

5.1 ESP32 BOARD



ESP32 is a low-cost, low-power Microcontroller with an integrated Wi-Fi and Bluetooth. It is the successor to the ESP8266 which is also a low-cost Wi-Fi microchip albeit with limited vastly limited functionality.

It is an integrated antenna and RF balun, power amplifier, low-noise amplifiers, filters, and power management module. The entire solution takes up the least amount of printed circuit board area. This board is used with 2.4 GHz dual-mode Wi-Fi and Bluetooth chips by TSMC 40nm low power technology, power and RF properties best, which is safe, reliable, and scale-able to a variety of applications.

The ESP32 is engineered to handle a wide spectrum of connectivity and processing requirements within a single integrated chip. Its dual-core Tensilica Xtensa LX6 processor operates at clock speeds of up to 240 MHz, enabling the device to execute complex real-time tasks such as simultaneous Wi-Fi communication, sensor data acquisition, motor control, and MQTT message publishing without performance degradation. The chip integrates 520 KB of SRAM and supports external flash memory of up to 16 MB, providing ample memory headroom for sophisticated firmware. With 34 programmable GPIO pins supporting multiple communication interfaces including SPI, I2C, UART,

I2S, and CAN, the ESP32 can interface directly with virtually any peripheral or sensor module without requiring additional bridge hardware. Beyond its hardware capabilities, the ESP32 benefits from a mature and actively maintained software ecosystem.

ESP32 SPECIFICATIONS

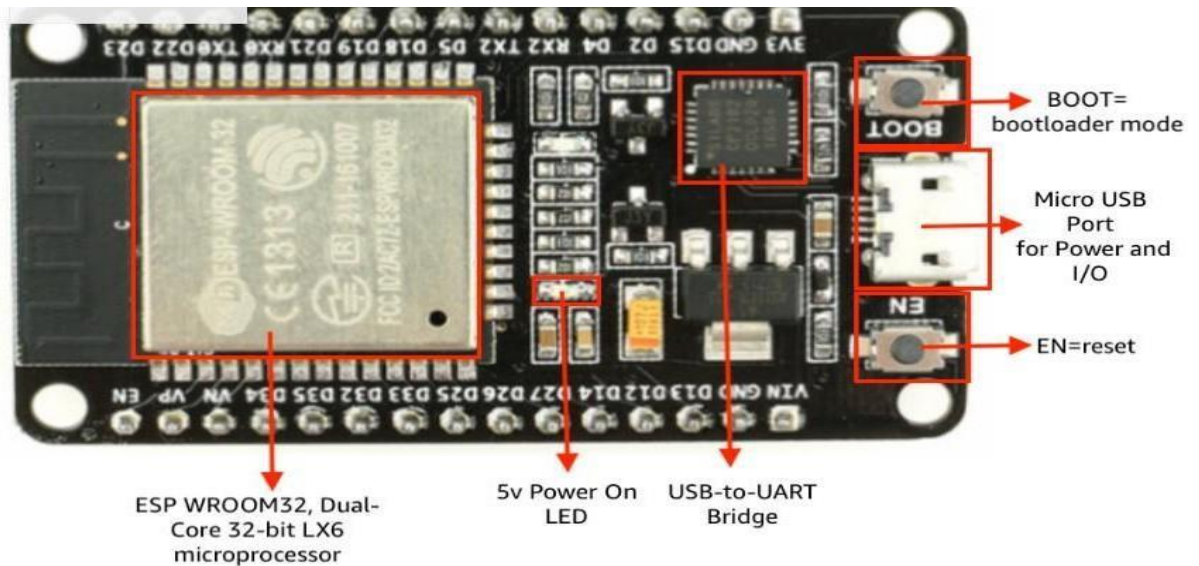


Fig 5.2 ESP32 specifications

- ROM: 448 KB, SRAM: 520 KB, Support up to: 16MB flash
- ESP WROOM32, Dual-Core 32-bit LX6 microprocessor
- Built-in CP21XX USB-to-UART(serial) Bridge
- Wi-Fi: 802.11b/g/n/e/i
- Bluetooth: v4.2 BR/EDR and BLE
- 2×8 -bit DACs [D26][D25]
- $9 \times$ touch sensors Note: Touch sensor 1 is [D0]. However, it's not available as a pin in this particular ESP32 development board (version with 30 GPIOs). GPIO 0 is available on the version with 36 pins. [D13][D12][D14][D24][D33][32] [D15][D2][D4]
- Internal Temperature Sensor

ESP32 BOARD GUIDE

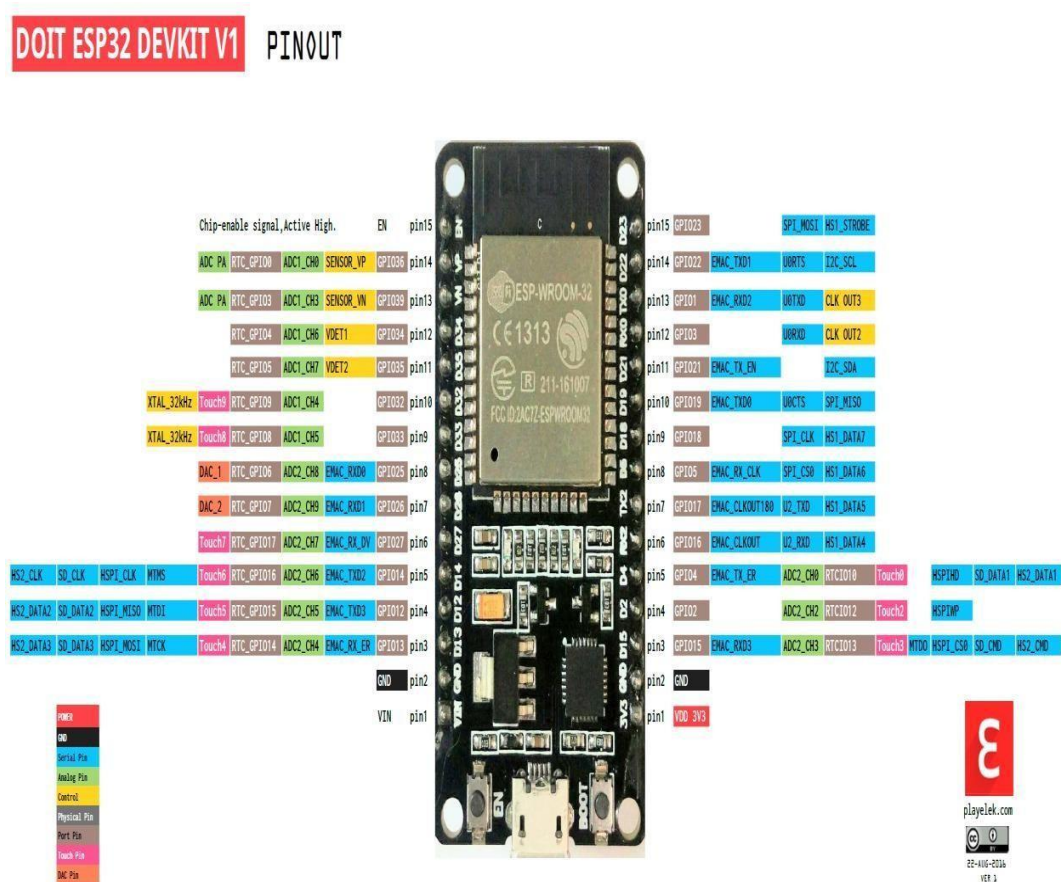


Fig 5.3 ESp32 board guide

The ESP32 supports three types of I/O modes with each GPIO Pin: Digital, Analog and Internal Sensors

Analog: Used to send/receive analog data using the following functions: `analogRead()`; `analogWrite()`;

Digital: Used to send/receive digital data using the following functions: digitalRead(); digitalWrite();

Internal Sensors: This mode allows us to fetch internal sensor data from the ESP32 itself. The three sensors available are as follows:

- Internal Temperature Sensor
- Hall Effect Sensor
- Touch Sensor

These sensors can be accessed by the following functions: `temperature_sens_read()` `hallRead()` `touchRead()`

ESP32 PIN DESCRIPTION

Power | Micro-USB, 3.3V, 5V, GND: Micro-USB: ESP32 can be powered through USB port. 5V: Regulated 5V can be supplied to this pin which will be again regulated to 3.3V by on board regulator, to power the board. 3.3V: Regulated 3.3V can be supplied to this pin to power the board. GND: Ground pins.

Enable | En: The pin and the button resets the microcontroller.

Analog Pins | ADC1_0 to ADC1_5 and ADC2_0 to ADC2_9: Used to measure analog voltage in the range of 0-3.3V. 12-bit 18 Channel ADC.

DAC Pins | DAC1 and DAC2: Used for Digital to Analog Conversion.

Input/Output Pins | GPIO0 to GPIO39: Totally 39 GPIO pins, can be used as input or output pins. 0V (low) and 3.3V (high). But pins 34 to 39 can be used as input only.

Capacitive Touch | T0 to T9: These 10 pins can be used as touch pins normally used for capacitive pads.

RTC GPIO Pins | RTCIO0 to RTCIO17: These 18 GPIO pins can be used to wake up the ESP32 from deep sleep mode.

Serial | Rx, Tx: Used to receive and transmit TTL serial data.

External Interrupts | All GPIO: Any GPIO can be used to trigger an interrupt.

PWM | All GPIO: 16 independent channels are available for PWM; any GPIO can be made to work as PWM through software.

VSPI | GPIO23 (MOSI), GPIO19 (MISO), GPIO18 (CLK), GPIO5 (CS): Used for SPI-1 communication.

HSPI | GPIO13 (MOSI), GPIO12 (MISO), GPIO14 (CLK), GPIO15 (CS): Used for SPI-2 communication.

IIC | GPIO21 (SDA), GPIO22 (SCL): Used for I2C communication.

AREF | AREF: To provide a reference voltage for input voltage.

ESP32 TECHNICAL SPECIFICATIONS

Microprocessor: Tensilica Xtensa LX6

Maximum Operating Frequency: 240 MHz

Operating Voltage: 3.3V

Analog Input Pins: 12-bit, 18 Channel

DAC Pins: 8-bit, 2 Channel

Digital I/O Pins: 39 (of which 34 is normal GPIO pin)

DC Current on I/O Pins: 40 mA

DC Current on 3.3V Pin: 50 mA

SRAM: 520 KB

Communication: SPI(4), I2C(2), I2S(2), CAN, UART(3)

Wi-Fi: 802.11 b/g/n

Bluetooth: V4.2 – Supports BLE and Classic Bluetooth

INPUT/OUTPUT

There are totally 39 digital Pins on the ESP32 out of which 34 can be used as GPIO and the remaining are input only pins. The device supports 18-channels for 12-bit ADC and 2-channel for 8-bit DAC. It also has 16 channels for PWM signal generation and 10 GPIO pins supports capacitive touch features. The ESP32 has multiplexing feature, this enables the programmer to configure any GPIO pin for PWM or other serial communication through program. The ESP32 supports 3 SPI Interface, 3 UART interface, 2 I2C interface, 2 I2S interface and also supports CAN protocol.

- **3 UART interface:** The ESP32 supports 3 UART interface for TTL communication. This would require 3 sets of Rx and Tx pins. All the 6 pins are software configurable and hence any GPIO pin can be programmed to be used for UART.
- **External Interrupt:** Again since the ESP32 supports multiplexing any GPIO pin can be programmed to be used as an interrupt pin.
- **GPIO23 (MOSI), GPIO19(MISO), GPIO18(CLK) and GPIO5 (CS):** These pins are used for SPI communication. ESP32 supports two SPI, this is the first set.
- **GPIO13 (MOSI), GPIO12(MISO), GPIO14(CLK) and GPIO15 (CS):** These pins are used for SPI communication. ESP32 supports two SPI, this is the second set.
- **GPIO21(SDA), GPIO22(SCL):** Used for IIC communication using Wire library.
- **Reset Pin:** The reset pin for ESP32 is the Enable (EN) pin. Making this pin LOW, resets the microcontroller.

Powering the ESP32

There are total three ways by which you can power your ESP32 board:

Micro USB Jack: Connect the mini USB jack to a phone charger or computer through a cable and it will draw power required for the board to function.

5V Pin: The 5V pin can be supplied with a Regulated 5V, this voltage will again be regulated to 3.3V through the on-board voltage regulator. Remember ESP32 operated with 3.3V only.

3.3V Pin: If you have a regulated 3.3V supply, then you can directly provide this to the 3.3V pin of the ESP32.

ESP32 APPLICATIONS

- Prototyping of IoT devices
- Low power battery operated applications
- Network projects

- Easy to use for beginner level DIYers and makers.
- Projects requiring Multiple I/O interfaces with Wi-Fi and Bluetooth functionalities.

5.2 Smoke Sensor (MQ-2 / MQTT)

5.2.1 Introduction to MQ-2 Smoke Sensor

The MQ-2 gas and smoke sensor is a widely used electrochemical sensor capable of detecting a broad range of combustible and toxic gases including LPG, propane, butane, methane, alcohol, hydrogen, smoke, and carbon monoxide. It is built around a heated tin dioxide (SnO₂) semiconductor sensing element enclosed within a stainless-steel mesh housing that protects the element from direct wind exposure while allowing gas to diffuse freely through it.

In clean air, oxygen molecules adsorb onto the surface of the SnO₂ layer, creating a high-resistance depletion barrier. When target gases come into contact with the sensing element, they react with the adsorbed oxygen ions, releasing electrons back into the SnO₂ conduction band and causing a measurable decrease in resistance. This resistance change is proportional to the gas concentration and is converted into a corresponding analog output voltage by the module circuitry. The sensor provides both an analog output (AO) for continuous concentration monitoring and a digital output (DO) that switches to a logic LOW when the detected gas level crosses a user-defined threshold set by the onboard sensitivity potentiometer.

The sensor requires a preheat period of at least 24 hours during initial deployment and approximately 20 seconds of warm-up time before each measurement session for stable and reliable readings. In this project, the analog output of the MQ-2 is connected to an ADC-capable GPIO pin of the ESP32, which samples the voltage and maps it to a gas concentration value. The digital output is connected to a separate GPIO pin configured as a digital interrupt, allowing the ESP32 to respond instantaneously to threshold crossing events without polling overhead.

5.2.2 MQ-2 Sensor Specifications

- Operating Voltage: 5V DC
- Heater Resistance: 33 Ohm ($\pm 5\%$)
- Heater Power Consumption: less than 800 mW

- Detectable Gas Concentration Range: 200 ppm to 10,000 ppm
- Output: Analog (AO) and Digital (DO) with adjustable threshold
- Operating Temperature: -20°C to 50°C
- Preheat Time: Greater than 24 hours for first use; 20 seconds per session

5.2.3 MQTT Integration

MQTT (Message Queuing Telemetry Transport) is a lightweight publish-subscribe messaging protocol built on top of TCP/IP, specifically designed for constrained devices and low-bandwidth, high-latency network environments. It operates on a broker-based architecture where a central MQTT broker manages all message routing between publishers and subscribers. This makes it significantly more efficient than HTTP-based communication for IoT sensor data transmission, as it eliminates the overhead of repeated HTTP request-response cycles.

In this project, the ESP32 acts as an MQTT client using the PubSubClient library. It establishes a persistent TCP connection to a cloud-hosted MQTT broker such as HiveMQ or a locally hosted Mosquitto broker over the Wi-Fi network. The ESP32 firmware continuously samples the MQ-2 analog output, converts the ADC reading to a gas concentration estimate, and publishes it as a structured JSON payload to a predefined MQTT topic at regular intervals. A separate MQTT alert topic is published immediately when the digital output of the sensor triggers a threshold alarm, ensuring the user is notified in real time without waiting for the next scheduled reading.

The user's monitoring application, such as MQTT Dash, Cayenne IoT, or a custom Node-RED dashboard, subscribes to the same topics and displays live gas concentration readings, trend graphs, and alarm notifications on a smartphone or laptop. MQTT's Quality of Service (QoS) levels allow the system to guarantee message delivery even over unreliable wireless links, making it well suited for safety-critical alert applications such as fire and gas hazard monitoring in a surveillance robot system.

5.2.4 Advantages of MQ-2 Sensor

- Wide detection range covering multiple combustible and toxic gases
- Provides both analog concentration output and digital threshold alarm

- Low cost and widely available for prototyping and deployment
- Simple interface requiring only one ADC pin and one digital GPIO pin
- Adjustable sensitivity threshold via onboard potentiometer
- Suitable for indoor and outdoor environments within the specified temperature range



Fig 5.4 MQ-2 Smoke Sensor

5.3 Flame Sensor

The flame sensor module is an electronic device designed to detect the presence of fire or an open flame by sensing the infrared (IR) radiation emitted during combustion. All flames emit a characteristic spectrum of electromagnetic radiation, with a significant portion falling in the near-infrared range between 760 nm and 1100 nm. The flame sensor exploits this property using an IR photodiode that is specifically sensitive to this wavelength range, making it highly selective for flame detection while being largely unaffected by ordinary ambient light sources such as fluorescent tubes or sunlight at normal intensities.

The module consists of an IR photodiode and an onboard LM393 dual comparator IC. When IR radiation from a flame strikes the photodiode junction, it generates a photocurrent proportional to the received IR intensity. This photocurrent develops a voltage across a load resistor, which is fed into the LM393 comparator. The comparator compares this input voltage against a reference threshold voltage set by the onboard potentiometer. When the flame-induced voltage exceeds the reference, the comparator output switches to a digital LOW signal at the DO (digital output) pin, indicating flame detection. The module simultaneously outputs a continuously varying analog voltage at the AO (analog output) pin, which reflects the real-time intensity of the detected IR radiation and allows the microcontroller to estimate the proximity and size of the flame.

The detection angle of the sensor is approximately 60 degrees and the effective detection range extends up to 1 metre depending on the intensity of the flame. The module operates on a supply voltage of 3.3V to 5V, making it directly compatible with the ESP32 without any level-shifting circuitry. In this project, the digital output is connected to a GPIO pin of the ESP32 configured as a digital interrupt, enabling the microcontroller to respond to flame detection events immediately. The analog output is simultaneously sampled by the ESP32 ADC to assess flame intensity and proximity, allowing the system to prioritise and grade the severity of the detected fire hazard before transmitting an alert.

5.3.1 Flame Sensor Specifications

- Operating Voltage: 3.3V to 5V DC
- Detectable Wavelength Range: 760 nm to 1100 nm (near-infrared)
- Detection Angle: Approximately 60 degrees
- Detection Range: Up to 1 metre depending on flame intensity
- Comparator IC: LM393 dual comparator
- Output Type: Digital (DO) active LOW and Analog (AO) continuous voltage
- Response Time: Less than 15 microseconds
- Operating Temperature: -25°C to 85°C

5.3.2 Advantages of Flame Sensor

- Extremely fast response time of less than 15 microseconds enables near-instantaneous fire detection
- Wide operating voltage range (3.3V to 5V) provides direct ESP32 compatibility
- Dual output (digital alarm and analog intensity) enables both threshold detection and proximity estimation
- Small form factor and lightweight, ideal for mounting on mobile robot platforms
- Adjustable sensitivity potentiometer allows fine-tuning for different deployment environments

5.3.3 Application in This Project

In this surveillance robot project, the flame sensor is mounted on the front face of the chassis to maximise its forward field of view. The digital output is connected to an interrupt-capable GPIO pin of the ESP32, allowing the firmware to immediately suspend normal operation and execute a fire alert routine upon flame detection, without any polling delay. The ESP32 simultaneously reads the analog output voltage to assess the intensity of the detected flame, classifying it as a minor flame (candlelight or small fire) or a major flame (large open fire) based on configurable ADC thresholds.

Upon flame detection, the ESP32 publishes both the alarm status and the analog intensity value as a JSON payload to a dedicated MQTT topic. The monitoring application receives this alert and displays a flame warning notification with intensity level on the operator's device, enabling the operator to assess the fire risk and take appropriate action. The flame sensor operates in conjunction with the MQ-2 smoke sensor to provide a complementary dual-layer fire hazard detection system, improving the reliability of fire detection and minimising false alarms.

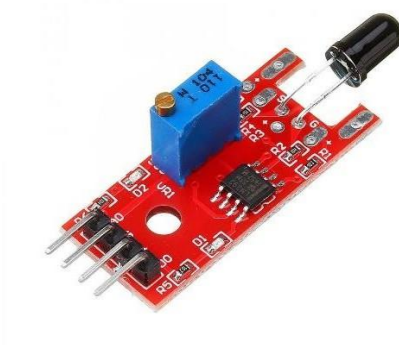


Fig 5.5 Flame Sensor Module

5.4 GPS Module (NEO-6M)

A GPS (Global Positioning System) module is an electronic device used to determine the precise geographical location of an object anywhere on Earth. It works by receiving signals from multiple satellites orbiting the Earth and calculating the exact latitude, longitude, altitude, and time of the receiver. GPS modules are widely used in navigation systems, vehicle tracking, marine monitoring, and location-based services.

GPS modules are commonly integrated with microcontrollers such as Raspberry Pi Pico, Arduino, or ESP32 to develop tracking and monitoring systems. They provide real-time location information that can be transmitted to monitoring stations or mobile devices using wireless communication technologies. Due to their high accuracy and reliability, GPS modules are widely used in maritime safety systems, vehicle tracking systems, and disaster management applications.

With the increasing demand for smart monitoring and navigation technologies, GPS modules play a crucial role in modern IoT-based tracking systems by providing continuous location updates and enabling real-time monitoring.

A GPS module is an electronic device that receives signals from satellites to determine the geographical location of a receiver. The module calculates position using signals from multiple satellites in the Global Positioning System. The position information is usually provided in the form of latitude and longitude coordinates.

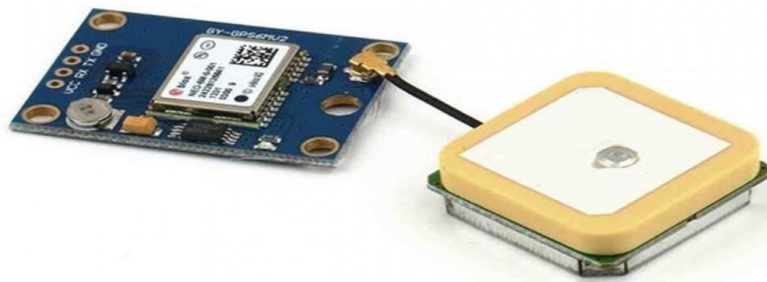


Fig 5.6 GPS NEO-6M Module

All GPS satellites continuously transmit signals containing their position and time information. The GPS receiver collects these signals and calculates its position using a process called trilateration.

GPS MODULE WORKING PRINCIPLE

A GPS module works by receiving signals from a group of satellites orbiting the Earth. At least four satellites are required to determine an accurate position.

Each satellite transmits signals that contain information about its location and the exact time the signal was sent. The GPS receiver measures the time taken for the signal to reach the receiver and calculates the distance from each satellite. By combining the distances from multiple satellites, the receiver determines the exact location of the device.

The GPS module then sends the calculated position data to the microcontroller using serial communication.

GPS MODULE CONSTRUCTION

The construction of a GPS module consists of several main components.

- 1. GPS Antenna:** The antenna receives signals transmitted by GPS satellites. A high-sensitivity antenna improves signal reception and accuracy.
- 2. GPS Receiver Chip:** The receiver chip processes the satellite signals and calculates the position coordinates such as latitude, longitude, altitude, and time.
- 3. Crystal Oscillator:** This component maintains precise timing required for accurate satellite signal processing.
- 4. Communication Interface:** The module communicates with microcontrollers using serial communication protocols such as UART.
- 5. Supporting Circuitry:** Additional components such as voltage regulators and filtering circuits ensure stable operation of the module.

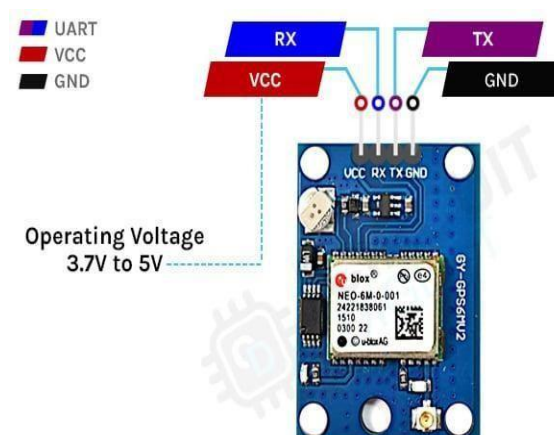


Fig 5.7 GPS Module Interface Diagram

WORKING

When the GPS module is powered on, it begins searching for signals from nearby satellites. Once the signals from at least four satellites are detected, the module calculates the device's position using trilateration.

The GPS module continuously updates the location information and sends the data to the microcontroller in the form of NMEA sentences through serial communication.

The microcontroller processes this data and can transmit the location information to a monitoring station using wireless communication technologies.

SPECIFICATIONS

- Operating voltage: 3.3 V – 5 V DC
- Position accuracy: Up to 2.5 meters
- Communication interface: UART serial communication
- Update rate: 1 Hz to 10 Hz
- Satellite support: GPS, GLONASS (depending on module)
- Sensitivity: Up to –165 dBm
- Operating temperature: –40°C to 85°C
- Antenna type: Built-in or external antenna

ADVANTAGES

- Provides accurate real-time location tracking
- Works anywhere with satellite coverage
- Low power consumption
- Easy integration with microcontrollers

- Supports navigation and monitoring applications
- Reliable operation in outdoor environments

APPLICATIONS

- Fisherman boat tracking systems
- Vehicle tracking systems
- Marine navigation systems
- Disaster management systems
- Military and defense navigation
- Smart transportation systems
- IoT-based monitoring systems

5.5 ESP32-CAM Module

The ESP32-CAM is a compact, low-cost camera module built around the ESP32-S chip combined with an OV2640 image sensor. It is one of the smallest Wi-Fi camera modules available, making it highly suitable for embedded surveillance and IoT imaging applications. The module features built-in Wi-Fi and Bluetooth connectivity, an onboard microSD card slot for local image storage, and a white LED flash. In this project, the ESP32-CAM is mounted on the robot chassis and provides live MJPEG video streaming over Wi-Fi, allowing the operator to remotely view the robot's surroundings in real time through a web browser.

The OV2640 camera sensor supports resolutions up to 2 megapixels (UXGA 1600x1200). The module internally compresses image frames in JPEG format before transferring them to the ESP32-S processor via a parallel data interface. The ESP32-S firmware initialises a Wi-Fi connection and hosts an HTTP server that streams the compressed JPEG frames as a Motion JPEG (MJPEG) video feed. Any device connected to the same network can access the live feed by navigating to the module's IP address in a browser.

5.5.1 ESP32-CAM Specifications

- Processor: ESP32-S, Dual-Core 32-bit, up to 240 MHz
- Camera Sensor: OV2640, up to 2 MP (UXGA 1600 x 1200)
- Flash Memory: 4 MB SPI Flash
- PSRAM: 4 MB for frame buffering
- Wi-Fi Standard: 802.11 b/g/n (2.4 GHz)
- Bluetooth: v4.2 BR/EDR and BLE
- Operating Voltage: 5V DC (3.3V regulated on-board)
- Operating Current: 180 mA average during video streaming
- Storage: MicroSD card slot (supports up to 4 GB)
- LED Flash: Built-in white LED
- Dimensions: 27 mm x 40.5 mm x 4.5 mm

5.5.2 Application in This Project

In this project, the ESP32-CAM operates independently from the main ESP32 controller. It is powered via the 5V rail from the buck converter and connected to the same Wi-Fi network as the main controller. The operator accesses the MJPEG video stream through a web browser on a smartphone or laptop to visually navigate the robot and assess the surrounding environment. This separation of video processing from sensor and motor control ensures that camera operations do not affect real-time sensor monitoring and motor response.



FIG 5.8 ESP32 Cam module

5.6 Robot Chassis

The robot chassis is the structural frame that supports and mechanically integrates all components of the surveillance robot. It provides a rigid platform for mounting the motors, wheels, controller boards, sensors, camera, GPS module, power electronics, and battery. For this project, a four-wheel drive (4WD) acrylic chassis is used, offering superior stability and traction on uneven surfaces compared to two-wheel configurations.

The 4WD chassis consists of a flat base plate with four DC gear motors fixed at each corner, each directly coupled to a rubber-tyred wheel. The gear reduction in each motor provides sufficient torque to carry the full weight of all mounted components while maintaining controlled and precise movement. Standard M3 mounting holes and brass standoffs allow all electronic modules to be securely fastened to the base plate.

5.6.1 Chassis Specifications

- Configuration: 4-Wheel Drive (4WD)
- Material: Acrylic base plate with aluminium motor mounts
- Base Plate Dimensions: 200 mm x 150 mm (approximate)
- Overall Height: 60 mm with wheels mounted
- Motor Type: DC Gear Motor, 6V to 12V, 150 to 200 RPM
- Wheel Diameter: 65 mm rubber-tyred wheels
- Maximum Load Capacity: Up to 1 kg
- Motor Driver Compatibility: L293D or L298N

5.6.2 Application in This Project

The four DC gear motors are controlled by the L293D motor driver IC, which receives PWM direction signals from the main ESP32 controller. All sensor modules, the ESP32 main board, ESP32-CAM, GPS module, smoke and flame sensors, buck converter, and battery pack are mounted on the chassis using standoffs and M3 screws. The low-profile acrylic construction keeps the centre of gravity low, preventing tipping during turns and traversal of inclined surfaces during surveillance operations.



Fig 5.9 Robot chassis

5.7 Buck Converter (DC-DC Step-Down)

A buck converter, also referred to as a step-down DC-DC converter, is a switching power supply that efficiently converts a higher input DC voltage to a lower regulated output DC voltage. Unlike linear voltage regulators that dissipate excess energy as heat, a buck converter uses an inductor, capacitor, diode, and switching transistor (MOSFET) to store and transfer energy with typical conversion efficiencies between 85% and 95%. This makes it the preferred power regulation solution for battery-powered mobile systems where maximising operational runtime is essential.

In this project, the LiPo battery pack supplies 11.1V. The ESP32 microcontroller, ESP32-CAM, GPS module, and sensor modules all require a regulated 5V supply. The buck converter steps down the battery voltage to 5V with high efficiency, significantly extending operational battery life compared to a linear regulator that would dissipate the voltage difference as heat.

5.7.1 Working Principle

The buck converter switches a MOSFET transistor on and off at high frequency, typically between 100 kHz and 1 MHz. When the switch is on, current flows from the input source through the inductor to the load, storing energy in the inductor's magnetic field. When the switch turns off, the energy stored in the inductor is released through the freewheeling diode to maintain continuous current to the load. An output capacitor smooths the voltage ripple. The ratio of switch-on time to the total switching period, called the duty cycle, determines the output voltage: $V_{out} = V_{in} \times \text{Duty Cycle}$.

5.7.2 Buck Converter Specifications

- Module: LM2596 based DC-DC Buck Converter
- Input Voltage Range: 4V to 35V DC
- Output Voltage Range: 1.25V to 30V DC (adjustable via potentiometer)
- Maximum Output Current: 3A continuous
- Conversion Efficiency: Up to 92%
- Switching Frequency: 150 kHz
- Output Voltage Ripple: Less than 30 mV
- Short Circuit Protection: Built-in
- Operating Temperature: -40 degrees C to 85 degrees C
- Dimensions: 43 mm x 21 mm x 14 mm

5.7.3 Application in This Project

The buck converter is configured to output a stable 5V by adjusting its onboard potentiometer and verifying the output with a multimeter before system power-up. The regulated 5V output powers the ESP32 main controller, ESP32-CAM, GPS module, MQ-2 smoke sensor, flame sensor, and ultrasonic sensor. The DC motors are powered directly from the battery through the L293D motor driver, bypassing the buck converter to allow full battery voltage operation for maximum torque.



Fig 5.10 Buck Converter Module

5.8 Battery

The battery is the sole power source of the surveillance robot, supplying energy to all electronic and electromechanical subsystems including the microcontrollers, sensors, camera, GPS module, and drive motors. For a mobile robotic platform, the battery must deliver sufficient voltage and discharge current while remaining lightweight and compact. In this project, a rechargeable 3-cell Lithium Polymer (LiPo) battery pack is used, selected for its high energy density, lightweight construction, and ability to sustain the high peak discharge currents demanded by the four DC drive motors.

A 3S LiPo configuration provides a nominal voltage of 11.1V (3.7V per cell) and a fully charged voltage of 12.6V (4.2V per cell). A capacity of 2200 mAh to 3000 mAh is selected to provide adequate operating time for extended surveillance missions. The battery directly powers the DC motors through the L293D motor driver and simultaneously feeds the buck converter, which supplies regulated 5V to all control and sensor electronics.

5.8.1 Battery Specifications

- Type: Lithium Polymer (LiPo), rechargeable
- Cell Configuration: 3S (3 cells in series)
- Nominal Voltage: 11.1V (3.7V per cell)
- Fully Charged Voltage: 12.6V (4.2V per cell)
- Minimum Safe Discharge Voltage: 9V (3.0V per cell)
- Capacity: 2200 mAh to 3000 mAh
- Maximum Continuous Discharge Rate: 20C to 30C
- Connector Type: XT60 or Deans (T-Plug)
- Charge Method: LiPo balance charger at 1C charge rate
- Weight: Approximately 160 g to 220 g

5.8.2 Safety and Handling

LiPo batteries require careful handling to prevent damage or fire hazards. The battery must never be discharged below 3.0V per cell, as deep discharge causes permanent cell damage. A low voltage cut-off or battery alarm is recommended to prevent over-discharge during operation. The battery must be charged using a dedicated LiPo balance charger to ensure all three cells charge equally to 4.2V. Damaged, swollen, or punctured LiPo batteries must not be used and must be disposed of following proper electronic waste procedures.

5.8.3 Application in This Project

The LiPo battery is mounted on the lower deck of the robot chassis for a low centre of gravity. The motor driver receives battery voltage directly for maximum motor torque. The buck converter

receives battery voltage and outputs regulated 5V to all control electronics. A voltage divider connected to the ESP32 ADC continuously monitors the battery voltage level. When voltage drops below the safe threshold, the firmware publishes a low-battery warning via MQTT, alerting the operator to end the mission and recharge the robot before the battery reaches the critical cut-off level.

Fig 5.11 battery



CHAPTER 6

SOFTWARE TOOLS

6.1 Arduino IDE Overview

The Arduino Integrated Development Environment (IDE) is a cross-platform application used for writing, compiling, and uploading programs to Arduino-compatible microcontroller boards, including the ESP32. The IDE provides a text editor, serial monitor, board/port selector, and a library manager. It supports C/C++ programming and provides built-in libraries for sensor interfacing, communication protocols, and hardware peripherals.

In this project, the Arduino IDE is used to develop all firmware for both the ESP32 main controller and the ESP32-CAM video streaming module. The IDE's library manager is used to install TinyGPSPlus, PubSubClient (MQTT), and ESP32 camera libraries required for this project.



Fig 6.1 Arduino IDE Logo

6.1.1 How Arduino IDE Works

When a sketch is uploaded, the IDE compiles it using the xtensa-esp32-elf-gcc compiler for the ESP32 target. The compiled binary is transferred to the ESP32 via USB using the esptool utility. The ESP32's bootloader receives the binary and stores it in flash memory. On power-up or reset, the ESP32 begins executing the stored sketch from the setup() function and enters the continuous loop() function thereafter.



Fig 6.1.1 Arduino IDE Interface

6.1.2 Key Libraries Used in This Project

WiFi.h — ESP32 Wi-Fi connectivity and web server hosting.

PubSubClient.h — MQTT client for smoke, flame, and GPS data publishing.

TinyGPSPlus.h — NMEA GPS sentence parsing for location data.

ESP32Servo.h — PWM-based servo and motor speed control.

esp_camera.h — ESP32-CAM camera initialization and frame capture.

6.1.3 Advantages of Arduino IDE for ESP32

Free and open-source with an active community and extensive library support.

Cross-platform: available on Windows, macOS, and Linux.

ESP32 board support available via Boards Manager (Espressif Systems package).

Built-in Serial Monitor for real-time debugging and GPS NMEA output viewing.

Simple single-click compile and upload workflow.

6.2 Working on Arduino IDE for ESP32

After installing the Arduino IDE and adding the ESP32 board package URL in Preferences > Additional Boards Manager URLs, the ESP32 DevKit V1 board is selected from Tools > Board. The

correct COM port is selected from Tools > Port. The USB driver (CP2102 or CH340) must be installed for the specific USB-to-Serial chip on the ESP32 board. After writing the sketch, clicking Upload compiles and flashes the firmware.

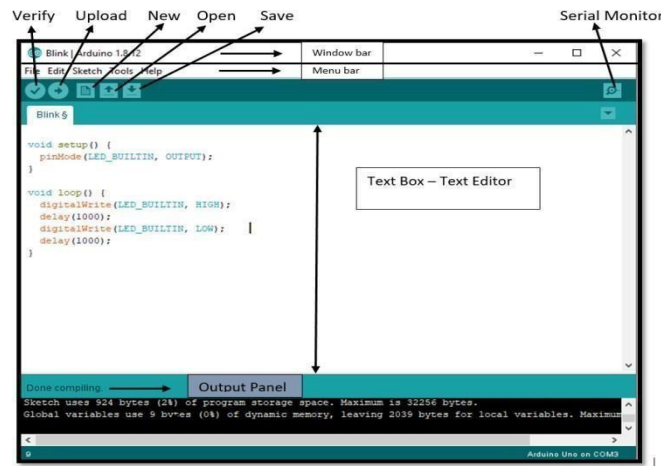


Fig 6.2 Arduino IDE Window

6.3 MQTT Platform (Cayenne / HiveMQ)

Cayenne by myDevices is an IoT application development platform that provides a graphical drag-and-drop dashboard interface. It supports MQTT communication with Arduino/ESP32-compatible devices and enables real-time data visualization, historical charting, threshold-based triggers, and mobile notifications. The platform uses the Cayenne MQTT protocol built on top of standard MQTT. Alternatively, HiveMQ public broker (broker.hivemq.com:1883) can be used for direct MQTT without account registration.

6.4 Tools Section

Board: ESP32 Dev Module | Partition Scheme: Default 4MB | Upload Speed: 921600 | Flash Mode: QIO | Port: COM3 (or /dev/ttyUSB0 on Linux)

6.5 Installation of Arduino IDE and ESP32 Board Package

1. Download Arduino IDE from <https://www.arduino.cc/en/software> and install.
2. Open Arduino IDE > File > Preferences.
3. In “Additional Boards Manager URLs” field, add:
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

4. Go to Tools > Board > Boards Manager, search “esp32”, and install the Espressif Systems package.
5. Install required libraries: Tools > Manage Libraries > search and install TinyGPSPlus, PubSubClient, ESP32Servo.



Fig 6.6 Download Arduino IDE page

6.7 Steps for Uploading Code to ESP32

1. Connect ESP32 to PC via USB cable.
2. Select “ESP32 Dev Module” under Tools > Board.
3. Select correct COM port under Tools > Port.
4. Open the surveillance robot sketch in Arduino IDE.
5. Click the Upload button (right arrow). The IDE compiles and flashes the firmware.
6. For ESP32-CAM: use an FTDI programmer, set GPIO0 to GND before upload. After upload, remove GPIO0-GND jumper and reset.
7. Open Serial Monitor at 115200 baud to verify Wi-Fi connection, MQTT broker connection, and GPS fix status.

Fig 6.7 IDE Window and Upload toolbar

SOURCE CODE

Code 1: Motor Control and Sensor Integration

```
#include <WiFi.h>

#include <WebServer.h>

#include <Wire.h>

#include <Adafruit_GFX.h>

#include <Adafruit_SSD1306.h>

#include <TinyGPS++.h>

#include <ESP_Mail_Client.h>


/***** WIFI *****/

const char* ssid = "Mr perfect";

const char* password = "perfect4707";


WebServer server(80);


/***** EMAIL *****/

#define SMTP_HOST "smtp.gmail.com"

#define SMTP_PORT 465

#define AUTHOR_EMAIL "sainathchowdary05@gmail.com"

#define AUTHOR_PASSWORD "qkih qbni nhtu zydj"

#define RECIPIENT_EMAIL "venkateshsagar4707@gmail.com"


/***** MOTOR PINS *****/

#define IN1 16

#define IN2 17
```

```

#define IN3 18

#define IN4 19

#define ENA 25

#define ENB 26


/***** ULTRASONIC *****/

#define TRIG 32

#define ECHO 33


/***** SENSORS *****/

#define MQ2 34

#define FLAME 27


/***** OLED *****/

#define SCREEN_WIDTH 128

#define SCREEN_HEIGHT 64

Adafruit_SSD1306 display(SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, -1);


/***** GPS *****/

TinyGPSPlus gps;

HardwareSerial gpsSerial(2);


bool autoMode = false;

int speedVal = 80;


String getLocation() {

```

```

if (gps.location.isValid())

    return "\nLocation:\nhttps://maps.google.com/?q=" +

        String(gps.location.lat(), 6) + "," +

        String(gps.location.lng(), 6);

return "\nLocation: GPS Searching...";
}

```

```

void sendAlert(String subject, String body) {

    ESP_Mail_Session session;

    session.server.host_name = SMTP_HOST;

    session.server.port = SMTP_PORT;

    session.login.email = AUTHOR_EMAIL;

    session.login.password = AUTHOR_PASSWORD;

    SMTP_Message message;

    message.sender.name = "SMART ROBOT ALERT";

    message.sender.email = AUTHOR_EMAIL;

    message.subject = subject;

    message.addRecipient("User", RECIPIENT_EMAIL);

    body += getLocation();

    message.text.content = body.c_str();

    SMTPSession smtp;

    if (!smtp.connect(&session)) return;

    MailClient.sendMail(&smtp, &message);

}

```

```

void setupMotors() {

```

```

pinMode(IN1,OUTPUT); pinMode(IN2,OUTPUT);

pinMode(IN3,OUTPUT); pinMode(IN4,OUTPUT);

ledcSetup(0,1000,8); ledcSetup(1,1000,8);

ledcAttachPin(ENA,0); ledcAttachPin(ENB,1);

ledcWrite(0,speedVal); ledcWrite(1,speedVal);

}

void forward() { digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
                digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW); }

void backward() { digitalWrite(IN1,LOW); digitalWrite(IN2,HIGH);
                 digitalWrite(IN3,LOW); digitalWrite(IN4,HIGH); }

void left()     { digitalWrite(IN1,LOW); digitalWrite(IN2,HIGH);
                 digitalWrite(IN3,HIGH); digitalWrite(IN4,LOW); }

void right()    { digitalWrite(IN1,HIGH); digitalWrite(IN2,LOW);
                 digitalWrite(IN3,LOW); digitalWrite(IN4,HIGH); }

void stopMotor() { digitalWrite(IN1,LOW); digitalWrite(IN2,LOW);
                  digitalWrite(IN3,LOW); digitalWrite(IN4,LOW); }

long getDistance() {
    digitalWrite(TRIG,LOW); delayMicroseconds(2);
    digitalWrite(TRIG,HIGH); delayMicroseconds(10);
    digitalWrite(TRIG,LOW);
    long d = pulseIn(ECHO,HIGH,30000);
    return d * 0.034 / 2;
}

```

```

void autonomousDrive() {
    long dist = getDistance();
    if (dist < 40) {
        stopMotor(); delay(300);
        int t = random(0,3);
        if (t==0) left(); else if (t==1) right(); else backward();
        delay(600);
    } else forward();
}

```

```

bool fireMailSent = false, gasMailSent = false;

```

```

void checkAlerts() {
    int gas = analogRead(MQ2);
    int flame = digitalRead(FLAME);
    if (flame==LOW && !fireMailSent) {
        sendAlert("FIRE DETECTED","Fire detected by Smart Surveillance Robot!");
        fireMailSent = true;
    }
    if (flame==HIGH) fireMailSent = false;
    if (gas>3500 && !gasMailSent) {
        sendAlert("GAS LEAK DETECTED","Dangerous Gas Level Detected!");
        gasMailSent = true;
    }
    if (gas<3000) gasMailSent = false;
}

```

```

void updateOLED() {
    while (gpsSerial.available()) gps.encode(gpsSerial.read());

    int gas = analogRead(MQ2);

    int flame = digitalRead(FLAME);

    display.clearDisplay();

    display.setTextSize(1); display.setTextColor(WHITE);

    display.setCursor(0,0);

    display.println("WiFi Connected");

    display.println(WiFi.localIP());

    display.print("MQ2:"); display.println(gas);

    display.print("Flame:");

    display.println(flame==0 ? "FIRE!" : "SAFE");

    display.print("LAT:");

    display.println(gps.location.isValid()?String(gps.location.lat(),4):"Searching");

    display.print("LNG:");

    display.println(gps.location.isValid()?String(gps.location.lng(),4):"Searching");

    display.display();
}

```

```

void setup() {
    Serial.begin(115200);

    setupMotors();

    pinMode(TRIG,OUTPUT); pinMode(ECHO,INPUT);

    pinMode(MQ2,INPUT); pinMode(FLAME,INPUT);

    randomSeed(micros());
}

```

```

Wire.begin(21,22);

display.begin(SSD1306_SWITCHCAPVCC,0x3C);

gpsSerial.begin(9600,SERIAL_8N1,14,13);

WiFi.begin(ssid,password);

while (WiFi.status()!=WL_CONNECTED) delay(500);

server.on("/",[]){server.send(200,"text/html","<h2>Robot Dashboard</h2>");});

server.on("/manual",[]){autoMode=false;stopMotor();server.send(200,"text/plain","OK");});

server.on("/auto",[]){autoMode=true;server.send(200,"text/plain","OK");});

server.on("/f",[]){autoMode=false;forward();server.send(200,"text/plain","OK");});

server.on("/b",[]){autoMode=false;backward();server.send(200,"text/plain","OK");});

server.on("/l",[]){autoMode=false;left();server.send(200,"text/plain","OK");});

server.on("/r",[]){autoMode=false;right();server.send(200,"text/plain","OK");});

server.on("/s",[]){autoMode=false;stopMotor();server.send(200,"text/plain","OK");});

server.begin();

}

void loop() {

  server.handleClient();

  updateOLED();

  checkAlerts();

  if (autoMode) autonomousDrive();

}

```

Code 2: ESP32-CAM Web Server

```
#include "esp_camera.h"

#include <WiFi.h>

// Camera model selection - ESP_EYE has PSRAM
#define CAMERA_MODEL_ESP_EYE

#include "camera_pins.h"

const char* ssid = "Mr perfect";
const char* password = "perfect4707";

void startCameraServer();

void setupLedFlash(int pin);

void setup() {
    Serial.begin(115200);
    Serial.setDebugOutput(true);

    camera_config_t config;
    config.ledc_channel = LEDC_CHANNEL_0;
    config.ledc_timer = LEDC_TIMER_0;
    config.pin_d0 = Y2_GPIO_NUM; config.pin_d1 = Y3_GPIO_NUM;
    config.pin_d2 = Y4_GPIO_NUM; config.pin_d3 = Y5_GPIO_NUM;
    config.pin_d4 = Y6_GPIO_NUM; config.pin_d5 = Y7_GPIO_NUM;
    config.pin_d6 = Y8_GPIO_NUM; config.pin_d7 = Y9_GPIO_NUM;
```

```

config.pin_xclk    = XCLK_GPIO_NUM;
config.pin_pclk    = PCLK_GPIO_NUM;
config.pin_vsync   = VSYNC_GPIO_NUM;
config.pin_href    = HREF_GPIO_NUM;
config.pin_sccb_sda = SIOD_GPIO_NUM;
config.pin_sccb_scl = SIOC_GPIO_NUM;
config.pin_pwdn    = PWDN_GPIO_NUM;
config.pin_reset   = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.frame_size  = FRAMESIZE_UXGA;
config.pixel_format = PIXFORMAT_JPEG;
config.grab_mode    = CAMERA_GRAB_WHEN_EMPTY;
config.fb_location  = CAMERA_FB_IN_PSRAM;
config.jpeg_quality = 12;
config.fb_count     = 1;

if (config.pixel_format == PIXFORMAT_JPEG) {
    if (psramFound()) {
        config.jpeg_quality = 10;
        config.fb_count     = 2;
        config.grab_mode    = CAMERA_GRAB_LATEST;
    } else {
        config.frame_size = FRAMESIZE_SVGA;
        config.fb_location = CAMERA_FB_IN_DRAM;
    }
} else {

```

```

    config.frame_size = FRAMESIZE_240X240;
}

#ifdef(CAMERA_MODEL_ESP_EYE)

    pinMode(13, INPUT_PULLUP);
    pinMode(14, INPUT_PULLUP);
#endif

esp_err_t err = esp_camera_init(&config);

if (err != ESP_OK) {
    Serial.printf("Camera init failed: 0x%x\n", err);
    return;
}

sensor_t *s = esp_camera_sensor_get();

if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1);
    s->set_brightness(s, 1);
    s->set_saturation(s, -2);
}

if (config.pixel_format == PIXFORMAT_JPEG)
    s->set_framesize(s, FRAMESIZE_QVGA);

#ifdef(LED_GPIO_NUM)
    setupLedFlash(LED_GPIO_NUM);
#endif

```

```
WiFi.begin(ssid, password);

WiFi.setSleep(false);

while (WiFi.status() != WL_CONNECTED) {
    delay(500); Serial.print(".");
}

Serial.println("\nWiFi connected");

startCameraServer();

Serial.print("Camera Ready! Use http://");

Serial.print(WiFi.localIP());

Serial.println(" to connect");
}

void loop() {
    delay(10000);
}
```

CHAPTER 7

RESULTS

7.1 Model Diagram of the Project:

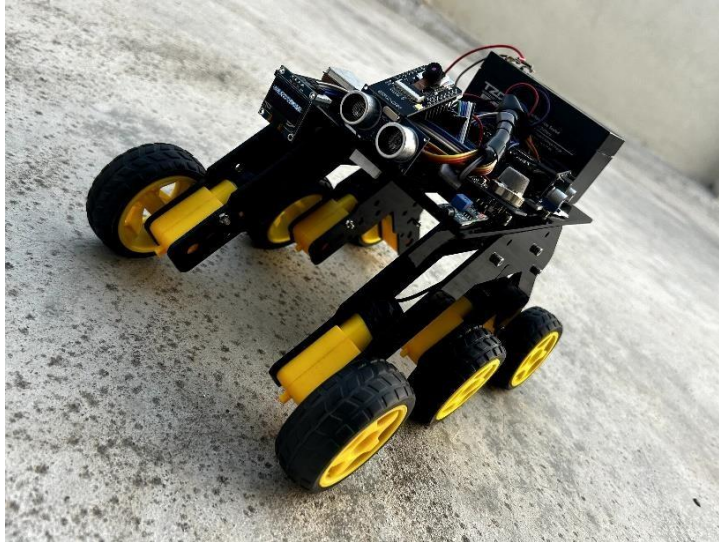
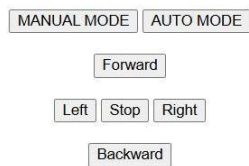


Figure 7.1 Working Model of the Surveillance Robot

SMART SURVEILLANCE ROBOT



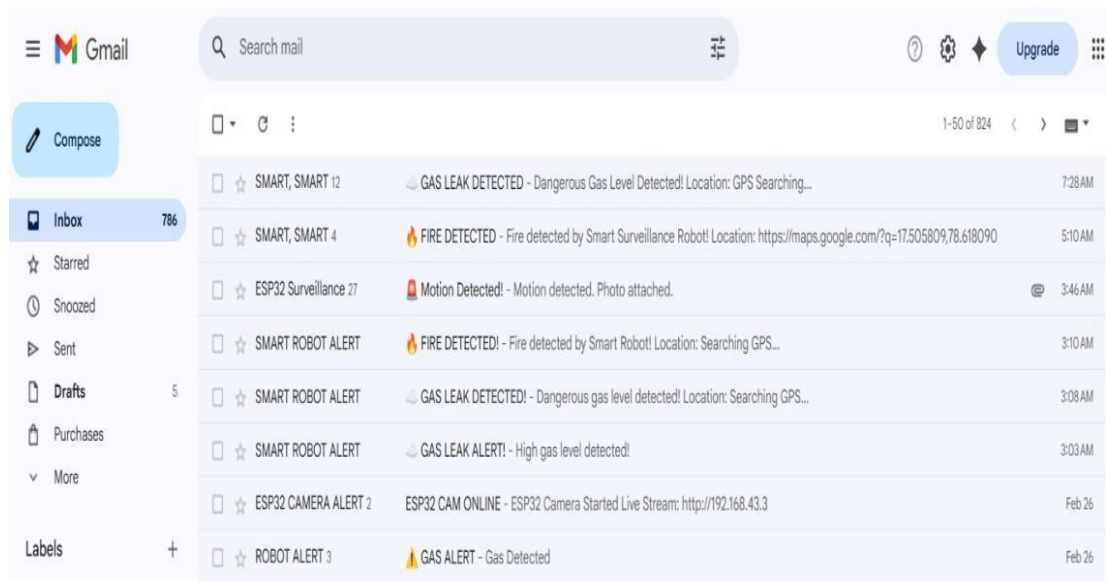


Figure 7.2 ON STATE — Robot Active with Live Video and Sensor Monitoring

7.3 ADVANTAGES

- **Enhanced Security:** Continuously monitors the environment, detects metal, smoke, and flame threats, and provides live GPS tracking for comprehensive security coverage.
- **Remote Operation:** Internet-based IoT control via ESP32 Wi-Fi allows monitoring and control from any location without range limitations.
- **Multi-Sensor Platform:** Combines video surveillance, obstacle avoidance, metal detection, smoke sensing (MQTT), flame detection, and GPS tracking in one robot.
- **Real-Time Alerts via MQTT:** Smoke and flame sensor data is pushed instantly to the user's mobile application, enabling prompt response to hazards.
- **Cost-Effective:** Built using affordable components (ESP32, L293D, MQ-2, NEO-6M), keeping total system cost minimal.
- **Scalability:** Future enhancements can add AI-based object recognition, additional environmental sensors, or solar power charging.

7.4 LIMITATIONS

- **Network Dependency:** The IoT control and MQTT communication require stable Wi-Fi. Performance degrades in areas with poor network coverage.
- **Limited Battery Life:** Continuous motor operation, video streaming, and multiple sensor polling drain the battery within a few hours.
- **GPS Indoor Performance:** GPS signal acquisition is poor indoors and in dense urban environments, limiting accurate tracking in enclosed spaces.
- **MQ-2 Warm-Up Time:** The MQ-2 smoke sensor requires a warm-up period of 20-30 seconds after power-on before providing reliable readings.
- **False Alerts:** The metal detector and flame sensor may produce occasional false positives from non-hazardous metallic objects or intense light sources.

7.5 APPLICATIONS

- **Home and Office Security:** Continuous monitoring of premises after hours, with instant MQTT alerts for smoke, fire, or intrusion.
- **Bomb Detection:** Metal detector enables scanning of public spaces, event venues, and sensitive areas for hidden metallic explosives or weapons.
- **Disaster Search and Rescue:** Can be sent into collapsed structures or disaster zones to survey conditions and transmit GPS location of survivors or hazards.
- **Industrial Fire Safety:** Monitors factories, chemical plants, and warehouses for smoke and flame hazards with real-time MQTT alert delivery to safety teams.
- **Border and Perimeter Patrol:** GPS tracking enables deployment for automated perimeter patrol with live location reporting to a central command center.
- **Educational Platform:** Demonstrates ESP32 IoT, MQTT, GPS, and multi-sensor robotics for engineering students learning embedded systems and automation.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

The Multi-Purpose IoT Based Surveillance Robot developed using the ESP32 microcontroller, ESP32-CAM, L293D motor driver, ultrasonic sensor, metal detector, smoke sensor (MQ-2/MQTT), flame sensor, and GPS module (NEO-6M) has demonstrated a practical, efficient, and cost-effective solution for automated multi-hazard surveillance. The system successfully integrates locomotion, real-time video monitoring, obstacle avoidance, metal detection, fire and smoke monitoring with instant MQTT alerts, and live GPS location tracking into a single mobile platform.

The ESP32's dual-core architecture and integrated Wi-Fi enable concurrent execution of motor control, sensor monitoring, MQTT communication, and GPS data parsing without performance bottlenecks. The ESP32-CAM provides live MJPEG video streaming to any browser, giving the operator real-time visual situational awareness. The MQTT-based smoke and flame alert system ensures that hazards are reported to the user instantly, even while the robot is operating autonomously.

The project fulfills all its core objectives: remote IoT control, live video surveillance, obstacle avoidance, metal detection, smoke/flame hazard detection with MQTT notifications, and GPS location tracking. It represents an interdisciplinary blend of embedded systems, IoT communication, robotics, and environmental sensing. The system can serve as a reliable foundation for commercially deployable surveillance solutions in security, defense, industrial safety, and disaster response domains.

8.2 FUTURE SCOPE

The Multi-Purpose IoT Surveillance Robot presented in this project is a functional prototype that opens numerous avenues for future enhancements and commercial deployment.

In future versions, the system can be integrated with cloud IoT platforms such as AWS IoT Core or Google Cloud IoT to enable centralized multi-robot monitoring, historical data logging, and predictive maintenance analytics. AI-based computer vision can be implemented using TensorFlow Lite on the ESP32 or an edge computing board (NVIDIA Jetson Nano) to enable automated intruder detection, facial recognition, and anomaly identification from the video feed.

4G/LTE modules (such as SIM7600) can replace the Wi-Fi dependency, enabling internet connectivity anywhere with cellular coverage and eliminating the need for a local Wi-Fi network. This would allow truly remote operation in open fields, forests, or disaster zones. The GPS capability can be enhanced with RTK (Real-Time Kinematic) GPS for centimeter-level accuracy, enabling precise autonomous navigation along pre-programmed patrol routes.

Additional sensors such as a temperature/humidity sensor (DHT22), a CO2 sensor, or a radiation detector can be added to the sensor array to expand the robot's environmental monitoring capabilities. Integration with emergency response systems (fire stations, police dispatch, hospital systems) via API can automate hazard reporting and reduce response times significantly.

With further research, this multi-purpose surveillance robot can evolve into a nationally deployable platform contributing to smart city security infrastructure, industrial safety compliance, border management, and disaster preparedness systems.

BIBLIOGRAPHY

- [1.] www.electronicsforu.com
- [2.] N. Nikhil, P. Shashank Reddy, K. Raju, R. Srinivasa Rao, “IoT Based Smart Multi-Application Surveillance Robot,” IRJMETS. DOI: <https://www.doi.org/10.56726/IRJMETS42292>
- [3.] International Research Journal of Engineering and Technology (IRJET). Available: <https://www.irjet.net/>
- [4.] International Journal of Computer Applications, March 2015.
- [5.] International Journal of Emerging Technology and Advanced Engineering (IJETA), ISSN 2250-2459, Volume 5, Issue 10, October 2015. Available: www.ijetae.com
- [6.] TEXAS Instruments Linear IC’s Manual — Reference for L293D Motor Driver specifications.
- [7.] SIGNETICS Digital IC’s Manual — Reference for digital logic interfacing with ESP32.
- [8.] Espressif Systems, “ESP32 Technical Reference Manual,” Available: <https://docs.espressif.com/>
- [9.] u-blox AG, “NEO-6 GPS Module Datasheet,” Available: <https://www.u-blox.com/>
- [10.] HiveMQ, “MQTT Protocol Documentation,” Available: <https://www.hivemq.com/mqtt/>